Software Engineering 2
Lösung der Übung: Automatisierter UI-Test

Dr. F.-K. Koschnick, Sybit GmbH

Copyright © 2024 Dr. F. K. Koschnick, Sybit GmbH. All rights reserved.

Aufgabe (Story): Automatisierter UI-Test

Als qualitätsbewusster Entwickler möchte ich ein WebDriver-Framework haben, mit dem ich automatisierte UI-Tests von Webapplikationen schreiben kann.

Verifiziere, dass
- der WebDriver über einen Chrome-Browser die UI-Tests steuert
- der WebDriver eine Methode get hat, mit der er den Browser veranlasst, eine Url aufzurufen
- der WebDriver eine Methode click hat, mit der ein beliebiges klickfähiges WebElement geklickt werden kann
- der WebDriver eine Methode input hat, mit der beliebige Zeichen in ein beliebiges Input-WebElement eingegeben werden können
- der WebDriver eine Methode getTextFromElement hat, mit der der Text eines beliebigen WebElements abgerufen werden kann
- der WebDriver eine Methode pageContains hat, mit der der WebDriver prüfen kann, ob ein beliebiger Text in der Webseite enthalten ist
- Ein Unit-Test existiert, mit dem der WebDriver mit einer beliebigen Webseite (z.B. www.google.de) getestet wird

Technische Bemerkung:
Verwendet das Projekt in Vorlesung Testen/UITesting und implementiert die entsprechenden Methoden (die Methodenrümpfe sind schon da).
Bitte in Eclipse ein neues Projekt UITesting dafür anlegen. Die Übungsaufgabe als Zip-Datei in das Repository einchecken.

Projektstruktur und build.gradle

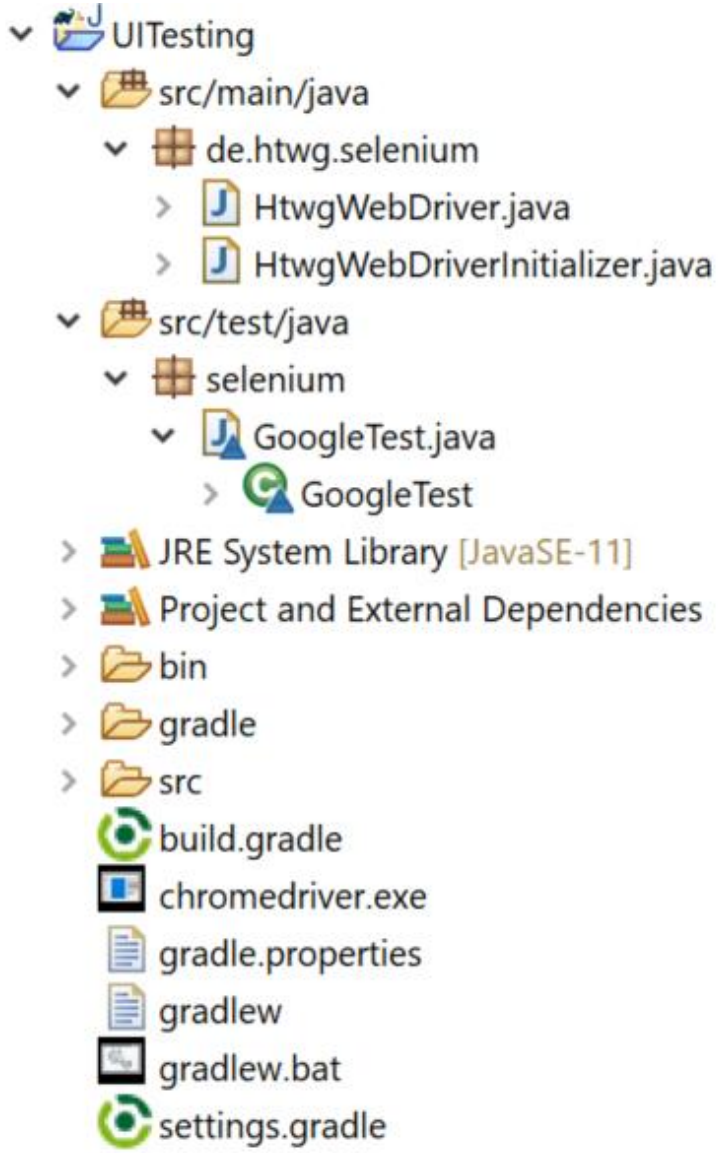

```
apply plugin: 'java'
apply plugin: 'eclipse'

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.seleniumhq.selenium:selenium-chrome-driver:4.21.0‘
    implementation 'org.seleniumhq.selenium:selenium-java:4.21.0‘

    testImplementation 'org.junit.jupiter:junit-jupiter-api:5.10.2‘
    testRuntimeOnly 'org.junit.jupiter:junit-jupiter-engine:5.10.2‘
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}


test {
    useJUnitPlatform()
    testLogging {
        events "passed", "skipped", "failed"
    }
}
```



WebDriver-Initializer wird von der Vorlesung übernommen


```java
package de.htwg.selenium;

import java.util.ArrayList;
import java.util.List;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class HtwgWebDriverInitializer {

private WebDriver webDriver;

public HtwgWebDriverInitializer() {
    ChromeOptions chromeOptions = setOptions("");
    this.webDriver = new ChromeDriver(chromeOptions);
}

public HtwgWebDriverInitializer(String arguments) {
    ChromeOptions chromeOptions = setOptions(arguments);
    this.webDriver = new ChromeDriver(chromeOptions);
}
```

```java
    private ChromeOptions setOptions(String arguments) {
        System.setProperty("webdriver.chrome.driver", "chromedriver.exe");
        ChromeOptions chromeOptions = new ChromeOptions();
        chromeOptions.setAcceptInsecureCerts(true);
        List<String> argumentList = new ArrayList<>();
        argumentList.add("start-maximized");
        if (arguments != null && !"".equalsIgnoreCase(arguments)) {
            String[] argArray = arguments.split(";");
            for (String argument : argArray) {
                argumentList.add(argument);
            }
        }
        chromeOptions.addArguments(argumentList); // e.g.: --headless or --disable-extensions
        return chromeOptions;
    }
    public WebDriver getWebDriver() {
        return webDriver;
    }
    protected void finalize() {
        try {
            if( webDriver != null) webDriver.quit();
        }catch(Exception e) {
            //do nothing
        }
    }
}
```

HtwgWebDriver

```
package de.htwg.selenium;

import java.util.List;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

public class HtwgWebDriver {
   private WebDriver webDriver;

   public HtwgWebDriver() {
      HtwgWebDriverInitializer wi = new HtwgWebDriverInitializer();
      this.webDriver = wi.getWebDriver();
   }

   public HtwgWebDriver(String arguments) {
      HtwgWebDriverInitializer wi = new HtwgWebDriverInitializer(arguments);
      this.webDriver = wi.getWebDriver();
   }

   public void get(String url) {
      webDriver.get(url);
   }
```

Im Prinzip besteht jede Methode aus dem Finden des Webelements oder der Webelemente mittels xPath und dann wird die entsprechende Operation (Klick, Texteingabe, etc.) auf dem gefundenen Element ausgeführt.

```
   public void switchToIFrame(int iFrame) {
      webDriver.switchTo().frame(iFrame);
   }

   public void switchBackToMain() {
      webDriver.switchTo().defaultContent();
   }

   public void click(String xpath) throws Exception {
      List<WebElement> elem = webDriver.findElements(By.xpath(xpath));
      elem.forEach(e -> {try {e.click();} catch (Exception error) { } });
   }
   public void input(String xpath, String content) {
      WebElement webElem = webDriver.findElement(By.xpath(xpath));
      webElem.sendKeys(content);
   }
   public String getTextFromElement(String xpath) {
      WebElement webElem = webDriver.findElement(By.xpath(xpath));
      return webElem.getText();
   }
   public boolean pageContains(String text) {
      return webDriver.getPageSource().contains(text);
   }
   public void close() {
      webDriver.manage().deleteAllCookies();
      webDriver.quit();
   }
   public void implicitlyWait(long milliSeconds) {
      try {
         Thread.sleep(milliSeconds);
      } catch (InterruptedException e) {
         e.printStackTrace();
      }
   }


}
```

Die Click-Methode ist etwas kompliziert, da es vorkommen kann, dass mehr als ein Element gefunden wird (bei Google gibt es zwei Such-Buttons, einer ist immer unsichtbar). Damit das funktioniert, werden alle Elemente durchiteriert. Nimmt man anstelle von *findElements* die Methode *findElement*, so wird nur das erste Element, das gefunden wird zurückgegeben. Ist auch möglich.

Zugehöriger JUnit-Test, der www.google.de testet


```
package selenium;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertTrue;

import de.htwg.selenium.HtwgWebDriver;

class GoogleTest {

private static HtwgWebDriver htwgWebDriver;

@BeforeEach
static void setUpBeforeTest() throws Exception {
   htwgWebDriver = new HtwgWebDriver(); // "--headless"
}

@AfterEach
static void tearDownAfterTest() throws Exception {
   htwgWebDriver.close();
}

@Test
void testGoogle() throws Exception {
   htwgWebDriver.get("https://www.google.de");
   htwgWebDriver. implicitlyWait(1000);
   htwgWebDriver.click("//*[@id=\"L2AGLb\"]");
   htwgWebDriver.input("//*[@name='q']", "test");
   htwgWebDriver.click("//*[@name='btnK']");
   htwgWebDriver.implicitlyWait(2000);
   assertTrue(htwgWebDriver.pageContains("Stiftung Warentest"));
}
}
```

Wenn man *"--headless"* als Argument in den Konstruktor packt, läuft der Browser ohne Fenster, also nur im Hintergrund. So würde man automatisierte Tests auf einem CI-Server laufen lassen.

Zustimmungsbutton für Nutzungsbedingungen von Google clicken.

Suchtextfeld von Google

Suchbutton von Google